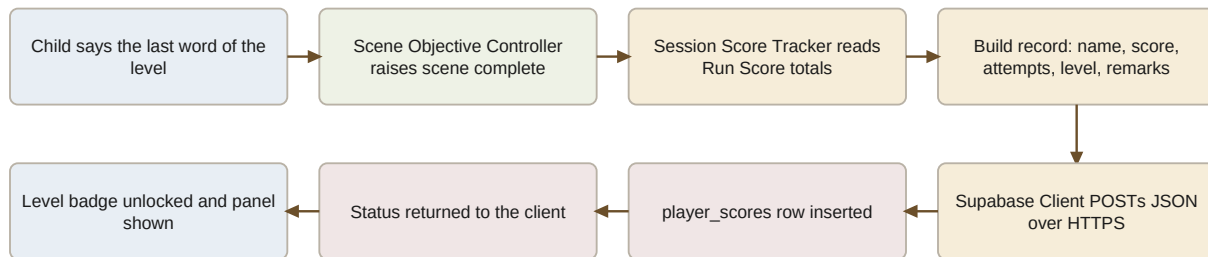


PART IV. END-TO-END INTEGRATION DESIGN

15 points

Process 1 — Save session result (online path)

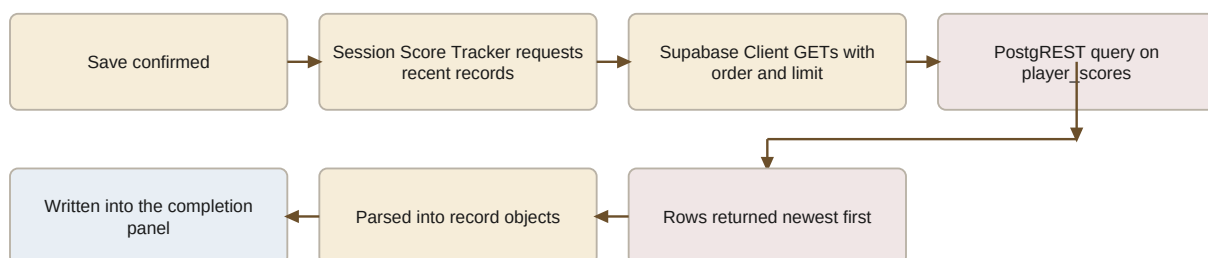


Item	Description
Trigger	The child correctly says the final spoken objective of the level
Source component	Scene Objective Controller (scene-complete event)
Data generated / sent	player_name, score, attempts, level, remarks as a JSON body. player_id and created_at are deliberately omitted so the database supplies them
Processing component	Session Score Tracker builds the record; Supabase Client performs the HTTPS POST
Destination	Supabase player_scores table, plus a local PlayerPrefs write to unlock the next level
Stored / retrieved data	One inserted row, echoed back by the API so the client can confirm what was stored
Expected response	HTTP 2xx with the inserted row; the client treats any other code as a failure
User output	The level-complete panel shows this run's score, and the next node on the journey map becomes available

Failure handling

Failure	What happens
No internet connection on the device	The request fails and the error is logged. The completion panel still shows the run's score from Run Score, and the level still unlocks, because unlocking is local. The session result is lost rather than queued — an offline queue is listed as planned work
Credentials missing or invalid (401)	The client detects the missing config before sending and reports a clear message naming the config asset instead of failing silently
Insert rejected by Row Level Security (403)	Treated as a save failure; the panel still displays the local score so play is never blocked
Scene complete fires twice	The tracker saves only once per run, so a repeated event cannot create duplicate rows

Process 2 — Retrieve and display recent results



Item	Description
Trigger	A successful save; the fetch runs immediately afterwards so the child sees the result in context
Source component	Session Score Tracker
Data generated / sent	A GET request with <code>select=*, order=created_at.desc</code> and a row limit (five by default)
Processing component	Supabase Client, which attaches the anon key headers and parses the JSON array
Destination	Back into the level-complete panel in the game UI
Stored / retrieved data	The five most recent session records, newest first
Expected response	HTTP 200 with a JSON array; an empty array is valid and means no sessions have been saved yet
User output	The completion panel lists the recent results underneath this run's score

Failure handling

Failure	What happens
Request fails or times out	The recent-results area is left empty and the error is logged; this run's own score is unaffected because it comes from Run Score, not from the fetch
Empty array returned	Not an error. It means nothing has been saved yet, and the panel simply shows no previous results
Record is missing expected fields	Unity's JSON parser leaves them at their default values, so a malformed row renders as a blank entry rather than crashing the panel

PART V. REPOSITORY AND IMPLEMENTATION STRUCTURE

10 points

Repository name	Group33_DatabaseConnectivity
Repository link	https://github.com/c23-2075-433-droid/Group33_DatabaseConnectivity
Visibility	Public
Development platform	Unity 6 (6000.6.2f1), 2D Universal Render Pipeline, C#
Target platform	Android (device speech recognition); Unity Editor used for development with a keyboard fallback
README status	Present and current — covers setup, database configuration, scene build order and controls
.gitignore status	Present — excludes Library/, Temp/, Logs/, builds, IDE files, and the credentials asset
Git LFS	Not used. The largest committed assets are PNG sprites well under GitHub's limit, so LFS would add setup cost without benefit
Branching approach	Single main branch with small, descriptive commits. Short-lived feature branches are planned once parallel work on separate chapters begins